

Django Vue Lab

Лекция 11. Безопасность веб-приложений

Черепанов И.Н. Филиппьев В.А.

Содержание

Уязвимости на стороне пользователя	4
Same-Origin Policy	6
CSRF	21
XSS	28
Header Injection и UI Redressing	42
Авторизация и данные	48
Парольные политики	66
White-Box анализ	77

Почему это важно

Безопасность – не отдельная кнопка в конце проекта, а свойство архитектуры, кода, настроек сервера и привычек команды.

- web используется не так как было задумано.
- браузер выполняет код страниц
- пользовательские данные нельзя считать безопасными
- frontend можно изменить в DevTools
- HTTP-запрос можно повторить вручную
- открытые сессии автоматически прикладывают cookies
- авторизация должна проверяться на сервере

Уязвимости на стороне пользователя

Клиентская сторона

Клиентская сторона – это не только HTML, CSS и JavaScript. Это среда, где пользователь, браузер, открытые сессии и чужие сайты взаимодействуют между собой.

Браузер

|

- +-- Same-Origin Policy
- +-- Cookies
- +-- CSRF
- +-- XSS
- +-- Header Injection
- +-- UI Redressing
- +-- AAA и логика приложения

Same-Origin Policy

Origin

Origin – это комбинация:

```
scheme + host + port
```

Примеры разных origin:

```
https://example.com  
http://example.com  
https://api.example.com  
https://example.com:8443
```

Путь URL не входит в origin.

Same-Origin Policy

Same-Origin Policy, SOP – политика ограничения видимости данных для скриптов в рамках определенного источника.

SOP защищает пользователя от чтения личных данных чужими сайтами.

```
https://evil.example  
    не может прочитать ответ  
https://bank.example/api/profile
```


Что SOP запрещает

- читать ответы fetch с другого origin
- читать DOM окна с другим origin
- читать cookies другого сайта
- читать localStorage другого origin
- обращаться к закрытым данным iframe другого сайта

Что SOP не запрещает

SOP не означает, что запрос нельзя отправить.

- форму можно отправить на другой сайт
- картинку можно загрузить с другого сайта
- скрипт можно подключить с CDN
- ссылку можно открыть
- браузер может приложить cookies к запросу

Отсюда возникает CSRF.

CORS

CORS – Cross-Origin Resource Sharing.

Это способ сервера явно разрешить браузеру читать cross-origin ответ.

```
Access-Control-Allow-Origin: https://app.example.com  
Access-Control-Allow-Credentials: true
```

CORS – не защита API от всех клиентов. Это политика для браузера.

Cookies

Cookies

Cookies – данные, которые сервер отправляет клиенту, а браузер затем автоматически отправляет с новыми запросами к этому серверу.



Берегите cookies: часто именно они подтверждают сессию пользователя.

Свойства cookies

- ограниченный срок жизни
- область действия включает путь и домен
- HttpOnly защищает от чтения через `document.cookie`
- Secure разрешает отправку только по HTTPS
- SameSite ограничивает отправку cookie с чужих сайтов
- Path ограничивает путь
- Domain расширяет область на поддомены

Сессии и токены

Session cookie:

- удобно для Django templates и session auth
- нужна CSRF-защита
- cookie лучше делать `HttpOnly`, `Secure`, `SameSite`

Bearer token:

- удобно для API и мобильных клиентов
- при краже токена атакующий действует как пользователь
- `localStorage` повышает риск при XSS

Жизненный цикл сессии

Сессия безопасна только если защищены все этапы ее жизни.

- генерация: идентификатор должен быть случайным и непредсказуемым
- передача: только по HTTPS, cookie с Secure
- хранение на клиенте: cookie с HttpOnly и SameSite
- хранение на сервере: ограниченный срок жизни и возможность отзыва
- завершение: logout должен инвалидировать сессию

После входа и повышения привилегий идентификатор сессии лучше обновлять, чтобы снизить риск session fixation.

Сессии в Django

Django session – серверное состояние, связанное с браузером через cookie.

```
Browser cookie: sessionid=abc123
      |
      v
Django session store: abc123 -> {"_auth_user_id": "7"}
```

В cookie обычно хранится не пользователь целиком, а ключ сессии. Данные сессии лежат в выбранном backend: базе данных, cache, signed cookie или другом storage.

Как работает request.session

Упрощенный порядок обработки запроса:

HTTP request

- > SessionMiddleware читает sessionid
- > загружает данные сессии
- > request.session становится доступен view
- > view меняет session
- > middleware сохраняет изменения и Set-Cookie

Если view вызывает `request.session["cart"] = ...`, Django помечает сессию как измененную и сохраняет ее в конце ответа.

AuthenticationMiddleware

AuthenticationMiddleware использует сессию, чтобы заполнить `request.user`.

```
session["_auth_user_id"]  
-> загрузка User из БД  
-> request.user
```

Поэтому порядок middleware важен: SessionMiddleware должен быть раньше AuthenticationMiddleware.

Middleware как цепочка

Middleware – слой обработки вокруг view.

```
request
-> SecurityMiddleware
-> SessionMiddleware
-> CsrfViewMiddleware
-> AuthenticationMiddleware
-> view
<- response
```

Middleware может изменить request, остановить запрос, добавить заголовки или изменить response.

Важные middleware безопасности

- SecurityMiddleware – HTTPS redirect, HSTS и другие security headers
- SessionMiddleware – загрузка и сохранение сессии
- CsrfViewMiddleware – проверка CSRF для небезопасных методов
- AuthenticationMiddleware – добавляет request.user
- XFrameOptionsMiddleware – защита от clickjacking

Отключать их стоит только если понятно, какая защита будет вместо них.

CSRF

CSRF

CSRF – Cross-Site Request Forgery, межсайтовая подделка запроса.

Если жертва авторизована на сайте, злоумышленник может попытаться заставить ее браузер отправить запрос на этот сайт.

```
Пользователь авторизован на bank.example  
Пользователь открывает evil.example  
evil.example отправляет POST на bank.example  
Браузер прикладывает cookies bank.example
```

Почему это работает

- cookies отправляются браузером автоматически
- пользователь уже авторизован
- SOP не запрещает сам факт отправки запроса
- сервер видит обычную сессию пользователя
- если нет дополнительной проверки, действие выполняется

Пример CSRF

```

```

```
<form action="https://bank.example/transfer" method="post">  
  <input name="to" value="attacker">  
  <input name="amount" value="10000">  
</form>  
<script>document.forms[0].submit()</script>
```

Проблема не в чтении ответа, а в выполнении действия от имени пользователя.

Защита от CSRF

- CSRF-токен в форме или заголовке
- SameSite=Lax или SameSite=Strict для cookies
- проверка Origin и Referer для небезопасных методов
- не менять состояние через GET
- повторная проверка важных действий

CSRF-токен

CSRF-токен – случайное значение, которое знает страница и сервер.

```
Cookie: sessionid=...  
Header: X-CSRFToken: random-value
```

Чужой сайт может отправить форму, но не может прочитать вашу страницу и узнать токен из-за SOP.

Django и Vue

Django включает CSRF-защиту для обычных форм и views.

- CsrfViewMiddleware проверяет небезопасные методы
- в шаблонах используется {% csrf_token %}
- для AJAX нужен заголовок X-CSRFToken
- @csrf_exempt использовать только с ясной причиной

```
await fetch("/api/orders/", {
  method: "POST",
  credentials: "include",
  headers: {
    "Content-Type": "application/json",
    "X-CSRFToken": getCookie("csrftoken"),
  },
  body: JSON.stringify(data),
})
```

Если используются bearer-токены без cookies, CSRF обычно не основная угроза.

XSS

XSS

XSS – Cross-Site Scripting, внедрение вредоносного кода в страницу, которую открывает пользователь.

```
комментарий пользователя -> HTML страницы -> выполнение script
```

XSS часто опаснее CSRF: вредоносный скрипт работает внутри доверенного origin.

Виды XSS

- Reflected XSS – вредоносный код отражается в ответе сразу
- Stored XSS – вредоносный код сохраняется на сервере
- DOM XSS – уязвимость в клиентском JavaScript

Что может сделать XSS

- читать DOM страницы
- отправлять запросы от имени пользователя
- красть токены из localStorage
- менять содержимое страницы
- выполнять действия в интерфейсе
- атаковать других пользователей через сохраненный контент
- распространяться как XSS-worm

Пример XSS

Пользователь оставляет комментарий:

```
<img src=x onerror="fetch('/api/me').then(r=>r.text()).then(alert)">
```

Если приложение вставит это как HTML, браузер выполнит обработчик onerror.

Каналы внедрения

Вредоносный код может попасть в страницу через разные каналы.

- ошибки браузера
- неправильное экранирование ", ', &, <, >
- HTML-атрибуты и обработчики событий
- ошибки кодировки
- HTML, BBCode, Markdown, вики-разметка
- SVG и загружаемые пользователем файлы

Любая разметка от пользователя должна быть либо экранирована, либо безопасно очищена.

Rich text

Rich text – форматированный пользовательский текст: жирный шрифт, ссылки, списки, картинки, цитаты.

Проблема: пользователю хочется дать оформление, но браузер понимает HTML как исполняемую среду.

```
<b>Привет</b>  
<img src=x onerror="alert(1)">  
<a href="javascript:alert(1)">link</a>
```

Нельзя просто сохранить и вывести rich text как готовый HTML без очистки.

BBCode

BBCode ограничивает набор тегов и выглядит безопаснее HTML.

```
[b]важно[/b]  
[url=https://example.com]ссылка[/url]
```

Но перед показом BBCode обычно превращается в HTML. Ошибка в парсере или проверке атрибутов снова приводит к XSS.

```
[url=javascript:alert(1)]нажми[/url]
```

Markdown

Markdown часто используют для комментариев, описаний задач и документации.

Риск зависит от настроек рендера:

- некоторые Markdown-движки разрешают HTML внутри Markdown
- ссылки могут вести на javascript: или data: URL
- картинки могут раскрывать приватные URL или использоваться для трекинга
- плагины таблиц, HTML-блоков и подсветки кода расширяют поверхность атаки

Безопасный рендеринг Markdown

Безопасная схема:

```
Markdown text  
-> Markdown renderer  
-> HTML sanitizer  
-> safe HTML
```

Правила:

- отключить сырой HTML, если он не нужен
- разрешить только безопасные теги и атрибуты
- проверять схемы ссылок: http, https, mailto
- добавлять rel="noopener noreferrer" для внешних ссылок
- не использовать v-html без санитайзера

Опасные места

- ручная сборка HTML строкой
- innerHTML
- Vue v-html
- отключение автоэкранирования в шаблонах
- Markdown без санитайзера
- загрузка SVG от пользователей
- вставка данных в JavaScript-код

Экранирование

Экранирование превращает данные в безопасный текст.

```
<script>alert(1)</script>
```

становится:

```
&lt;script&gt;alert(1)&lt;/script&gt;
```

Шаблоны Django и Vue по умолчанию экранируют текст.

Защита от XSS

- не вставлять пользовательский ввод как HTML
- использовать автоэкранирование шаблонов
- избегать `v-html` и `innerHTML`
- санитизировать HTML, если он действительно нужен
- валидировать и нормализовать ввод
- хранить сессионные cookies с `HttpOnly`
- включить Content Security Policy

Content Security Policy

CSP ограничивает, откуда можно загружать и выполнять ресурсы.

```
Content-Security-Policy:  
  default-src 'self';  
  script-src 'self';  
  object-src 'none';  
  base-uri 'self'
```

CSP не заменяет исправление XSS, но снижает ущерб.

Header Injection и UI Redressing

Header Injection

Header injection возникает, когда пользовательский ввод попадает в HTTP-заголовок без проверки.

```
http://target.example/away?<user_input>  
...  
Location: <user_input>
```

Если атакующий добавит перевод строки, он может попытаться внедрить новый заголовок.

Последствия Header Injection

- расщепление ответа
- фиксация сессии
- подмена поведения браузера
- установка нежелательных cookies
- перенаправление пользователя на вредоносный ресурс

Защита от Header Injection

- не вставлять пользовательский ввод в заголовки напрямую
- проверять URL перед редиректом
- использовать allowlist доменов для внешних переходов
- удалять управляющие символы CR и LF
- использовать стандартные функции фреймворка для redirect/response

UI Redressing

UI Redressing – атаки на восприятие интерфейса пользователем.

- изменение вида страницы
- наложение элементов
- скрытые iframe
- обман клика пользователя
- опасные стили и прозрачные элементы

Практический пример – clickjacking: пользователь думает, что нажимает одну кнопку, а действие уходит в другой интерфейс.

Защита от Clickjacking

- X-Frame-Options: DENY ИЛИ SAMEORIGIN
- CSP frame-ancestors 'self'
- не размещать критичные действия в iframe
- требовать подтверждение важных операций
- визуально отделять опасные действия

Авторизация и данные

Концепция AAA

AAA – базовая модель управления доступом.

- Authentication – установить личность пользователя
- Authorization – проверить разрешения
- Accounting – записать действия для контроля и расследования

```
login/password -> пользователь Иван  
permission check -> Иван может редактировать только свои заказы  
audit log -> Иван изменил заказ 15 в 12:30
```

Без accounting сложно понять, кто изменил данные, когда начался инцидент и какие объекты нужно восстановить.

Способы аутентификации

- HTTP Basic Authentication
- form-based login
- многофакторная аутентификация
- OpenID Connect и OAuth 2.0 login
- TLS client certificates
- session cookie
- bearer token

Выбор зависит от клиента: браузерное приложение, API, мобильное приложение, внутренний сервис.

Сообщения при входе

Сообщения об ошибках не должны помогать перебору учетных записей.

Плохо:

Ошибочный логин

Ошибочный пароль

Вы ввели пароль пользователя Joy

Лучше:

Логин или пароль неверны

Для пользователя сообщение должно быть понятным, но не должно раскрывать, что именно существует в системе.

Проверка прав на сервере

Frontend может скрыть кнопку, но не может быть источником безопасности.

Плохо:

кнопка "Удалить" скрыта во Vue

Нужно:

API DELETE /orders/10 проверяет владельца заказа

Любой API endpoint должен проверять права доступа.

Контроль доступа

Контроль доступа связывает три вещи:

- ресурсы: страницы, API endpoints, объекты БД, файлы
- роли: гость, пользователь, менеджер, администратор
- права доступа: читать, создавать, изменять, удалять, подтверждать

В проекте важно явно описать, какие роли существуют и какие ресурсы им доступны.

Матрица доступа

Простой способ проверить авторизацию – таблица ролей и действий.

Ресурс	Гость	Пользователь	Владелец	Админ
Заказ list	нет	свои	свои	все
Заказ edit	нет	нет	свои	все
Товар edit	нет	нет	нет	все

Если правило нельзя записать в такую таблицу, его будет сложно проверить в коде.

Горизонтальный и вертикальный доступ

Горизонтальная атака: пользователь получает доступ к данным другого пользователя того же уровня.

Иван открывает `/api/orders/16`, где 16 -- заказ Петра

Вертикальная атака: пользователь получает доступ к функциям более высокой роли.

обычный пользователь вызывает `/api/admin/users/5/ban`

IDOR

IDOR – Insecure Direct Object Reference.

Пользователь меняет идентификатор объекта и получает чужие данные.

```
/api/orders/15  
/api/orders/16
```

Защита: фильтровать queryset по текущему пользователю и проверять object-level permissions.

Распространенные ошибки доступа

- прямой вызов ресурса без проверки прав
- контроль доступа только по параметру запроса
- нарушение последовательности вызова
- контроль по местоположению или IP вместо роли
- проверка только в интерфейсе, но не в API

Пример: пользователь не видит кнопку оплаты, но может вручную вызвать `POST /api/orders/15/pay/`.

Нарушение последовательности

Некоторые операции безопасны только в правильном порядке.

1. создать заказ
2. выбрать доставку
3. оплатить
4. подтвердить

Если API позволяет вызвать шаг 4 без шагов 2 и 3, это ошибка бизнес-логики.

Контроль по параметру

✗ Опасно доверять параметрам, которые пришли от клиента.

```
{  
  "user_id": 7,  
  "role": "admin",  
  "price": 1  
}
```

Владелец, роль, цена и состояние процесса должны определяться на сервере.

Контроль по параметру: форма

✗ Плохой вариант: сервер доверяет скрытым полям формы.

```
<form action="/orders/15/update/" method="post">  
  <input type="hidden" name="user_id" value="7">  
  <input type="hidden" name="role" value="admin">  
  <input type="hidden" name="price" value="1">  
  <button>Сохранить</button>  
</form>
```

hidden скрывает поле только от обычного взгляда, но не защищает данные. Пользователь может изменить HTML в DevTools или отправить запрос вручную.

Контроль по параметру: правильно

Сервер должен брать критичные значения из доверенного контекста.

```
user_id -> из request.user  
role    -> из базы данных  
price   -> из товара/тарифа на сервере  
owner   -> проверяется по объекту в БД
```

Форма может передавать намерение пользователя, но не должна назначать ему права, владельца, цену или состояние бизнес-процесса.

SQL Injection

SQL injection возникает, когда пользовательский ввод становится частью SQL-кода.

```
User.objects.raw(f"SELECT * FROM users WHERE name = '{name}'")
```

Защита:

- использовать Django ORM
- использовать параметры запроса, а не f-string
- не собирать SQL руками без необходимости
- ограничивать права пользователя БД

Валидация данных

Валидация нужна на клиенте и на сервере.

- клиентская валидация **помогает UX**
- серверная валидация **защищает данные**
- нельзя доверять disabled, hidden, readonly
- нельзя доверять цене, роли или владельцу из frontend

Загрузка файлов

Файлы от пользователей опасны.

- ограничить размер
- проверять расширение и MIME type
- хранить вне исполняемых директорий
- генерировать имя файла на сервере
- не доверять SVG и HTML
- проверять доступ к приватным файлам

SSRF

SSRF – Server-Side Request Forgery.

Сервер заставляют сделать запрос туда, куда пользователь напрямую не имеет доступа.

```
POST /preview-url  
url=http://127.0.0.1:8000/admin/
```

Защита: allowlist доменов, запрет локальных адресов, таймауты.

Парольные политики

Плохие пароли

Плохой пароль легко подобрать или угадать.

- пустой пароль
- слишком короткий пароль
- пароль совпадает с логином
- переиспользование паролей на разных сайтах
- словарные слова
- пароль по умолчанию
- очевидные шаблоны: `qwerty`, `123456`, `password`

В Django стоит использовать встроенные `validators` паролей и не снижать их требования без причины.

Усечение пароля

Опасная ошибка – незаметно менять пароль пользователя перед проверкой.

- ограничить длину и отбросить хвост
- привести к одному регистру
- удалить часть символов из-за кодировки
- обрезать пробелы там, где они могли быть частью пароля

Если пароль был изменен системой до хеширования, его стойкость может резко снизиться.

Хранение пароля

Варианты от худшего к нормальному:

- открытый текст
- свой алгоритм шифрования
- простой хеш
- соленый хеш с медленной функцией

Django хранит пароли через password hashers и добавляет соль. Не нужно писать собственную схему хранения паролей.

Политика смены пароля

Смена пароля нужна не по расписанию ради расписания, а при риске компрометации.

- смена при инциденте
- смена при следующем логине после сброса
- требование старого пароля при обычной смене
- защита формы смены пароля от CSRF
- защита от перебора текущего пароля

Периодическая принудительная смена без причины часто приводит к более слабым паролям.

План на случай инцидента

Если пароль или сессия могли утечь, нужен понятный disaster plan.

- зафиксировать событие в логах
- уведомить пользователя
- сбросить активные сессии
- выдать ссылку на смену пароля с ограниченным сроком
- потребовать смену при следующем логине
- не отправлять новый пароль письмом

Восстановление пароля

Восстановление пароля часто слабее основного входа.

Опасные решения:

- простой секретный вопрос
- ответ, который можно подобрать или найти в соцсетях
- раскрытие данных пользователя через форму восстановления
- показ логина после правильного ответа
- одноразовая ссылка без срока жизни
- отправка нового пароля на почту или телефон

Безопаснее: одноразовая ссылка, короткий срок жизни, ограничение попыток, нейтральные сообщения и уведомление пользователя.

Логи

Логи нужны для расследования, но сами могут стать утечкой.

- логировать ошибки и важные действия
- не логировать пароли, токены, cookies
- не показывать traceback пользователю
- хранить логи с ограниченным доступом
- отслеживать подозрительные действия

Канал передачи данных

Пароль проходит не только через форму входа. На пути есть несколько участков.

- рабочая станция пользователя
- локальная сеть
- маршрутизатор
- провайдер
- reverse proxy
- хостинг
- приложение

На каждом участке возможны перехват, подмена или утечка.

Защита канала

- TLS/HTTPS для всего сайта
- Secure cookies
- HSTS после проверки корректной HTTPS-настройки
- challenge-response там, где нельзя передавать секрет напрямую
- двухфакторная аутентификация для критичных аккаунтов
- осторожность с публичными Wi-Fi и чужими устройствами

HTTPS

HTTPS защищает трафик между браузером и сервером.

- логины и cookies не уходят открытым текстом
- сложнее подменить страницу
- можно использовать Secure cookies
- HTTP/2 и современные браузерные функции часто требуют HTTPS

HTTPS не защищает от XSS, SQL injection и ошибок авторизации.

White-Box анализ

White-Box

White-box анализ – проверка системы с доступом к внутренней информации.

- исходный код
- конфигурации
- схема БД
- настройки сервера
- зависимости
- права доступа
- процесс разработки

Такой анализ помогает найти ошибки, которые плохо видны при обычном тестировании через интерфейс.

Операционная система

Что проверять на уровне OS:

- локальная политика безопасности
- права доступа к файлам и каталогам
- конфигурации сервисов
- интегральные проверки состояния системы
- patch management
- запущенные службы и открытые порты

Для web-проекта это относится к серверу, reverse proxy, systemd-сервисам, директориям media/static и секретам окружения.

Сеть

Что проверять в сетевом окружении:

- DNS-записи
- маршрутизацию
- доступность внутренних сервисов снаружи
- firewall/security groups
- TLS-сертификаты
- открытые административные интерфейсы

Ошибка в сети может открыть наружу то, что приложение считало внутренним.

Персонал

Безопасность зависит не только от кода.

- права доступа сотрудников
- доступ внештатных сотрудников
- обучение команды
- процесс выдачи и отзыва доступов
- запрет общих учетных записей
- разделение production и development доступов

Доступ должен выдаваться по необходимости и отзываться при изменении роли.

Анализ исходных кодов

Статический анализ:

- поиск опасных функций и паттернов
- проверка зависимостей
- поиск секретов в репозитории
- проверка конфигураций

Динамический анализ:

- тестирование работающего приложения
- проверка прав доступа через реальные запросы
- fuzzing форм и API
- проверка ошибок и логов

Примеры статических анализаторов

- SonarQube – платформа анализа качества кода, bugs, code smells и security hotspots
- bandit – ищет типовые проблемы безопасности в Python-коде
- semgrep – поиск уязвимых паттернов по правилам для разных языков
- ruff – быстрый анализ Python-кода, часть ошибок видна до запуска
- pip-audit – проверка Python-зависимостей на известные уязвимости
- gitleaks – поиск секретов в репозитории

Статический анализ хорошо запускать в CI до merge.

Примеры динамических анализаторов

- OWASP ZAP – сканирование работающего web-приложения
- Burp Suite – проху для ручного и автоматизированного тестирования
- sqlmap – проверка SQL injection
- ffuf – перебор путей, параметров и виртуальных хостов
- browser DevTools + ручные запросы – проверка прав доступа и состояния UI

Динамический анализ требует тестового стенда и осторожности: сканер отправляет реальные запросы к приложению.

Top-Down и Bottom-Up

Top-down анализ идет от архитектуры к деталям.

```
роли -> ресурсы -> endpoints -> проверки в коде
```

Bottom-up анализ идет от кода к общей картине.

```
view/service -> queryset -> permissions -> бизнес-правило
```

Оба подхода полезны: первый ищет пропущенные требования, второй – реальные ошибки реализации.

Безопасность в цикле разработки

Безопасность должна появляться на каждом этапе.

- техническое задание:
- архитектура:
- программирование:
- тестирование:
- релиз:

Если безопасность проверяется только перед релизом, исправления становятся дорогими.

Типовые уязвимости в учебных проектах

- любой пользователь видит все записи
- редактирование объекта по id без проверки владельца
- админские кнопки скрыты только во frontend
- DEBUG=True на сервере
- токены лежат в localStorage и есть XSS
- пользовательская строка вставляется через v-html
- секреты опубликованы на GitHub
- logout не завершает серверную сессию
- API позволяет перескочить шаг бизнес-процесса
- доступ к ресурсу контролируется только параметром user_id

Главное

- Не доверяйте клиенту
- Проверяйте права на сервере
- Экранируйте пользовательские данные
- Не отключайте CSRF без причины
- Используйте безопасные настройки cookies
- Не храните секреты в коде
- Думайте сценариями атак, а не списком настроек

Спасибо за внимание

Навык “Web разработка” повышается